

Embabel Agent Framework — Cheat Sheet

Embabel 1.0 learning demo · Print / PDF companion to docs/CHEATSHEET.md

Mental model

```
User/system input → Blackboard (typed)
  → Planner picks next @Action
  → Action: LLM / code / tools / subagents
  → Return posts types → REPLAN
  → @AchievesGoal return type → DONE
```

OODA: Observe blackboard → Orient conditions → Decide plan → Act

Annotations

Annotation	Meaning
@Agent	Spring + agent; description for selection
@EmbabelComponent	Reusable actions, not a full agent
@Action	Planner step; types drive GOAP
@AchievesGoal	Completing action finishes process
@Condition	Boolean fact (side-effect free)
@RequireNameMatch	Bind by name, not only type
@State	Scope actions; enable loops
@Export(remote=true)	Shell / MCP / A2A exposure
@LlmTool / @Tool	Custom tool / domain tool (needs attach)

@Action knobs

Attr	When
pre/post	Named conditions
cost/value	GOAP cost / Utility value
canRerun	May run more than once
outputBinding	Name return for RequireNameMatch
clearBlackboard	Looping state actions
description	Critical for Supervisor

LLM patterns

```
// Java
ai.withDefaultLlm().createObject(p, Story.class);
ai.withLlm(LlmOptions.withAutoLlm().withTemperature(0.7))
  .creating(Story.class).fromPrompt(p);
ai.withDefaultLlm().createObjectIfPossible(p,
Person.class);
```

Planners

Planner	Picks by	Best for
GOAP	Cheap typed path	Business workflows
UTILITY	Highest net value	Triage / explore
HYBRID	Gather + goal stop	Gather → synthesize
SUPERVISOR	LLM chooses tools	Flexible multi-step

Tools (per PromptRunner)

API	Does
withToolObject	Expose domain @Tools
withToolGroup(WEB)	Capability bundle
withTool(...)	Custom @LlmTool
Subagent...consuming	Agent-as-tool
ToolCallContext	Out-of-band; not in schema

Blackboard rules

- Latest of a type wins — unless named binding
- Return new type = post effect
- null / failed ifPossible → replan
- Conditions called often → pure
- State transitions hide prior states

```
// Kotlin
context.ai().withDefaultLlm().createObject<Story>(p)
context.ai().withDefaultLlm().createObjectIfPossible<Person>(p)
fromForm<PersonProfile>("Fill this in")
```

Invocation

```
AgentInvocation.builder(agentPlatform)
  .options(ProcessOptions.DEFAULT.withVerbosity(Verbosity.DEFAULT.withShowPrompts(true)))
  .build(ReviewedStory.class)
  .invoke(new UserInput("...")); // selects by goal return type
```

Java vs Kotlin

	Java	Kotlin
DTO	record	data class
Context	often Ai	often OperationContext
Create	Class arg	reified create/createObject
HITL	WaitFor.formSubmission	fromForm
Agents	annotations	annotations or DSL @Bean

Debug first steps

- First @Action entry
- createObject / fromPrompt
- Action return (blackboard post)
- @Condition / planner choice
- Goal @AchievesGoal

Don't forget

- Source order ≠ execution order
- @Tool needs withToolObject
- High HITL cost = fallback
- Supervisor needs action descriptions

- `StuckHandler` $\neq$ incomplete model

Source: docs/CHEATSHEET.md · Regenerate: scripts/generate-cheatsheet-pdf.sh · Companion: docs/TOP_10.md · docs/CHAPTER_SUMMARIES.md · docs/REVIEW_CIRCUIT.md